TileFlow: Spatial Neural Networks on AMD XDNA 2 NPU

Hardware-software co-design for close-to-metal neural network inference

Built with IRON/MLIR-AIE toolchain — Target: AMD Ryzen AI 9 HX 370 (Strix Point)

1. Introduction

Modern neural processing units (NPUs) are spatial dataflow computers: instead of a single CPU with caches and a register file, they expose a 2D array of small compute tiles, each with its own SRAM and SIMD units, connected by a programmable interconnect. This architecture is extremely efficient for *on-chip* computation — but only if the software maps the algorithm directly to the physical hardware.

TileFlow takes this literally. We design a neural network whose architecture is dictated by the physical tile layout of the AMD XDNA 2 NPU. The network has learnable parameters and non-linearities (ReLU), but its structure (number of layers, block grouping, buffer sizes) matches the hardware exactly.

Our main result is a **block-recurrent character language model** that generates Shakespeare-like text entirely on the NPU. The model has 32 layers grouped into 8 blocks of 4. Each block maps to a 4-stage pipeline on the NPU (one layer per compute row, 8 columns in parallel). Between blocks, the CPU applies normalisation, character-embedding injection, and residual connections — the same operations the model uses during training. The result: **89,600 characters/second** on 32 NPU tiles, with the trained weights producing *identical* computation on hardware — no approximations, no graph compilation, no operator fallback.

> **Key principle:** We design the network to match the hardware, not the other way around. Any architecture with learnable parameters and non-linearities can learn — so we choose the one that maximizes hardware utilization.

2. Background

This section provides the hardware and systems context that most ML engineers never need to think about — until they want to understand *why* certain hardware is fast (or slow) for their

models. If you've trained models with PyTorch or JAX and think of hardware as "a GPU with some VRAM," this section fills in the gap between that abstraction and the physical reality of a spatial processor like the XDNA 2 NPU.

2.1 Glossary of acronyms

The table below defines every acronym used in this paper. Each entry has an `id` so that occurrences in the text link back here. We group them by domain so you can revisit this section as a reference while reading.

Acronym	Stands for	What it means
Hardware & memory		
NPU	Neural Processing Unit	A dedicated accelerator for neural-network inference (and sometimes training), built into a laptop or phone chip.
APU	Accelerated Processing Unit	AMD's name for a single die that integrates CPU + GPU + NPU.
XDNA	(AMD brand name)	AMD's NPU architecture family. XDNA 2 is the second generation, found in "Strix Point" Ryzen AI chips.
AIE	AI Engine	The individual tile processor IP inside the XDNA NPU, originally designed by Xilinx (acquired by AMD).
SRAM	Static Random-Access Memory	Fast, on-chip memory (~1–2 ns access). Each compute tile has ~64 KB of SRAM. Expensive per bit, but extremely fast because it sits right next to the compute logic.
DRAM	Dynamic Random-Access Memory	The main system memory (8–64 GB). Much slower than SRAM (~50–100 ns access) but vastly cheaper per bit.
DDR	Double Data Rate	The interface standard for DRAM modules. "DDR memory" is the system RAM your laptop uses. In this paper, "DDR" means "host-side main memory."
DMA	Direct Memory Access	A hardware mechanism that copies data between memory regions *without using the CPU*. The NPU's shim tiles contain DMA engines that move data between DDR and tile SRAM.
PCIe	Peripheral Component Interconnect Express	The high-speed bus connecting the NPU to the rest of the system.
Compute concepts		

SIMD	Single Instruction, Multiple Data	A processor executes one instruction on a *vector* of values simultaneously. For example, multiplying 32 numbers in one clock cycle instead of one at a time. This is how GPUs and NPUs achieve massive throughput.
VLIW	Very Long Instruction Word	A processor design where each instruction encodes *multiple operations* to execute in parallel (e.g., a multiply, an add, and a load all in one cycle). Each AIE tile uses a VLIW core.
MMUL	Matrix Multiply (unit)	A hardware block dedicated to multiplying small matrices (e.g., 8×8 blocks of bfloat16). This is the workhorse of each AIE tile.
FIFO	First In, First Out	A queue where data is read in the same order it was written. The NPU uses hardware FIFOs ("ObjectFIFOs") to stream data between tiles, like Unix pipes between processes.
GEMM	General Matrix Multiply	The standard linear algebra operation C = A × B + C. Neural network layers are essentially sequences of GEMMs with non-linearities in between.
Numeric formats		
bf16 / bfloat16	Brain Floating Point, 16-bit	A 16-bit floating-point format with the same exponent range as float32 (8 bits) but less precision (7-bit mantissa vs 23-bit). Invented at Google Brain for ML training where range matters more than precision.
BFP16	Block Floating Point, 16-bit	An emulation mode on AIE2P that groups bf16 values into blocks sharing a common exponent. Enables efficient SIMD matmul in the MMUL unit with tile factor r = s = t = 8.
INT8	8-bit Integer	8-bit integer arithmetic, used for quantized inference. The NPU's peak in INT8 mode is 50 TOPS (double the bf16 rate).
Performance metrics		
FLOPS	Floating-point Operations Per Second	The standard measure of compute throughput. One multiply-add on two numbers counts as 2 FLOPs.
GFLOPS	Giga FLOPS (10^9)	Billions of floating-point operations per second.
TFLOPS	Tera FLOPS (10^{12})	Trillions of floating-point operations per second. The NPU's peak is 25 TFLOPS in bfloat16.
TOPS	Tera Operations Per Second	Like TFLOPS but for integer operations. Used for INT8 peak specs (50 TOPS for this NPU).

Machine learning		
MLP	Multi-Layer Perceptron	A neural network made of fully-connected (dense) layers. Each layer computes y = activation(x × W + b).
ReLU	Rectified Linear Unit	The activation function max(x, 0). Simple, cheap to compute, and widely used.
Toolchain & software		
IRON	(name, not an acronym)	AMD's Python API for programming AIE tiles at a high level: defining kernels, ObjectFIFOs, and tile placements.
MLIR	Multi-Level Intermediate Representation	A compiler framework (from the LLVM project) that represents programs at multiple abstraction levels. MLIR-AIE is the dialect that targets AIE hardware.
XRT	Xilinx Runtime	The userspace library and kernel driver that loads bitstreams onto the NPU and manages execution.
XCLBIN	Xilinx Container for Linux Binary	The compiled binary file that contains the NPU bitstream (tile configuration, routing, kernel code).
LLVM	Low Level Virtual Machine	A widely-used compiler infrastructure. Peano/LLVM-AIE is a fork that compiles C++ to AIE tile machine code.
BD	Buffer Descriptor	A hardware structure in the DMA engine that describes one data transfer: source address, size, stride pattern. The 10-bit size field (max 1024) is a constraint discussed in Section 4.3.

2.2 Key vocabulary (non-acronyms)

Several important terms in this paper are not acronyms but may be unfamiliar to readers who have worked above the hardware abstraction layer:

Term	What it means
Shim tile	The row of interface tiles at the bottom of the NPU array (row 0). "Shim" means a thin adapter layer — these tiles *bridge* two different worlds: the system's DDR memory (accessed via PCIe/AXI bus) and the NPU's internal tile interconnect. Each shim tile contains DMA engines that translate between host memory addresses and on-chip tile coordinates. They do no computation; they are pure data movers.
Tile	A self-contained processing unit with its own local memory (SRAM), instruction memory, and compute logic. Unlike CPU cores that share

	caches, each tile is an *independent computer* that communicates with neighbours through explicit data channels (FIFOs), not shared memory.
Bitstream	The compiled binary that programs the NPU's hardware configuration: which tiles run which code, how the interconnect routes data, and what DMA transfers to perform. Analogous to a GPU shader binary, but configuring a full spatial processor rather than a single shader core. Packaged in an XCLBIN file.
Kernel (NPU)	A C++ function compiled to run on a single tile. Not to be confused with an OS kernel or a CUDA kernel (which runs on thousands of GPU threads). An NPU kernel runs on exactly *one* tile's VLIW+SIMD core and has direct access to that tile's SRAM.
Pipeline	A technique where multiple stages of a task overlap in time, like an assembly line. While stage N processes item *i*, stage N+1 processes item *i−1*. The chess compiler pipelines the inner loops of our matmul kernel so that memory loads, multiplications, and stores overlap across iterations.
Register	The smallest, fastest storage inside a processor core — a few hundred bytes at most, accessible in a single clock cycle. The MMUL unit accumulates partial matrix products in registers before writing back to SRAM. Our fused kernel zero-initialises these accumulators in registers, avoiding a separate SRAM clear pass.
Memory wall	The growing gap between processor speed and memory speed. CPUs can execute arithmetic ~100× faster than DRAM can supply data. This is *the* fundamental reason NPUs (with data-local SRAM) can be dramatically faster than CPUs for the right workloads.
Interconnect	The on-chip wiring and switches that connect tiles to each other and to memory tiles. Unlike a CPU's shared bus, the NPU interconnect is *circuit-switched*: routes are configured at compile time, giving dedicated bandwidth with no contention.
Throughput vs Latency	**Latency** is how long one operation takes (e.g., one matmul = 1 μs). **Throughput** is how many operations complete per second (e.g., 24 tiles × 1000 matmuls = 24M matmuls/s). The NPU excels at throughput through massive parallelism, even if single-tile latency is similar to a CPU.
Double buffering	Using two buffers that swap roles each iteration: one is being read by the compute unit while the other is being filled by DMA. Also called "ping-pong buffering." This hides data transfer latency behind computation.

2.3 How CPUs execute neural networks (and why it's slow)

When you call `y = torch.relu(x @ W)` on a CPU, here is what actually happens at the hardware level:

1. The weight matrix `W` lives in DRAM. The CPU issues a load instruction, and the data travels through the **memory hierarchy**: DRAM → L3 cache → L2 cache → L1 cache → registers.
2. The CPU's SIMD units multiply a few rows at a time. The result goes back through the cache hierarchy to DRAM.
3. For the next layer, the process repeats: read the activation from DRAM, read the next weight matrix from DRAM, compute, write back.

The fundamental problem is the **memory wall**: modern compute units can perform arithmetic far faster than memory can supply data. A typical CPU can do ~1 TFLOPS of bf16 math, but its DDR bandwidth is ~50 GB/s. To keep the ALUs busy doing a 128×128 matmul, you need 128×128×2 = 32 KB of weight data delivered every ~1 μs — which is 32 GB/s just for one matrix. If you're running 4 layers, that's 128 GB/s, already exceeding the memory bandwidth. The CPU stalls waiting for data.

> **The memory wall in one sentence:** Arithmetic is cheap; moving data is expensive. The deeper you go in a neural network, the more time is spent shuffling data between memory levels, not doing useful math.

2.4 The spatial dataflow alternative

A **spatial architecture** takes a radically different approach. Instead of one fast processor with a deep cache hierarchy, it uses *many small processors* (tiles), each with a tiny but *extremely fast* local memory (SRAM). Data moves between tiles through dedicated hardware channels (FIFOs), not through shared caches.

Think of it like a factory assembly line versus a single master craftsman:

- **CPU (craftsman):** One highly skilled worker goes to the warehouse (DRAM) for each part, brings it to the workbench (registers), processes it, takes the result back to the warehouse, gets the next part. Most time is spent walking, not working.
- **NPU (assembly line):** Many workers sit at their stations with all their parts already on their desk (SRAM). A conveyor belt (FIFO) moves the workpiece from one station to the next. Nobody walks anywhere.

The key property is **data locality**: once data is loaded into a tile's 64 KB SRAM, it stays there for as many operations as you can do on it. There is no cache to get evicted from, no bus to contend for. If a 128×128 weight matrix (32 KB) fits in SRAM, you can multiply against it thousands of times at full speed — which is exactly what our recurrent MLP does.

2.5 Key hardware concepts used in this project

These concepts appear throughout the paper. You don't need to understand every transistor, but knowing these ideas will make the design choices clear:

Double buffering (ping-pong)

If a tile needs to *receive* new data while *computing* on data it already has, you need two buffers: one being filled by DMA while the other is being read by the compute unit. They swap roles each iteration. We use this for the activation buffers: buffer A holds the input, the tile computes into buffer B, then B becomes the input and A becomes the output.

Tiled matrix layout

A "row-major" matrix stores elements left-to-right, top-to-bottom: `[[a, b], [c, d]]` becomes `[a, b, c, d]`. The AIE matmul unit instead expects a **blocked (tiled) layout**: the matrix is divided into 8×8 sub-matrices, and each block is stored contiguously. This matches the MMUL hardware which multiplies 8×8 blocks in one operation. The `to_tiled()` function handles this conversion.

ObjectFIFOs and data movement

On a CPU, data movement is implicit — you `load` from an address and the cache hierarchy handles the rest. On the NPU, you must *explicitly* program every data transfer: "move 4 KB from DDR address X to tile (3, 2)'s input buffer." IRON's `ObjectFifo` abstraction makes this manageable: you declare a typed channel between a producer and a consumer, and the compiler generates the DMA configurations.

The invocation overhead problem

Every time the host CPU tells the NPU to run, there is a fixed overhead of ~120 µs for driver calls, instruction dispatch, and DMA setup. This is analogous to the overhead of launching a CUDA kernel on a GPU. If your actual compute takes only 1 µs (as it does for a single small matmul), you are spending 99% of the time on overhead. The solution is to do *lots of work per invocation* — hence the hardware loop that repeats thousands of matmuls before returning to the host.

2.6 How to read the rest of this paper

With this background, the rest of the paper should be accessible:

- **Section 3** describes the physical hardware: the tile grid, memory sizes, and interconnect.
- **Section 4** presents the neural network architecture: block-recurrent character language model mapped to the NPU pipeline.
- **Section 5** shows quality and throughput results.
- **Section 6** describes the software toolchain.
- **Section 7** maps the code structure to the concepts.

2.7 Neural network fundamentals (for hardware engineers)

If you work in hardware or systems and have heard "neural network" but never implemented one, this section gives you just enough to follow the architecture decisions in this paper.

What is a neural network?

A neural network is a function built by stacking **layers**. Each layer applies a linear transformation (matrix multiply) followed by a **non-linearity** (a simple element-wise function). For example:

```
y = ReLU(x @ W + b)      # one layer
```

Here `x` is the input vector, `W` is a learnable **weight matrix**, `b` is a learnable **bias vector**, and ReLU(z) = max(z, 0) is the non-linearity. Without the non-linearity, stacking layers would collapse to a single matrix multiply — the non-linearity is what gives depth its power. Stacking many such layers creates a "deep" network that can learn increasingly abstract representations of its input.

Language models: predicting the next character

A **language model** assigns probabilities to sequences of text. The simplest version predicts the next character (or word) given everything that came before. If the model has read "KING RICHAR", it should assign high probability to "D". At generation time, we sample from this probability distribution to produce text one character at a time.

The quality of a language model is measured by **cross-entropy loss** — roughly, how surprised the model is by each character in a held-out test set. Lower is better. The related metric **perplexity** = e^{loss} can be interpreted as "the model is as confused as if choosing uniformly among *N* options." A perplexity of 11 means the model is, on average, as uncertain as if it were picking among 11 equally likely characters (out of ~65 in our vocabulary).

Recurrence: memory across time

A **recurrent** network maintains a **hidden state** `h` that carries information from one time step to the next:

```
h = f(h, embed(char))     # update hidden state with new character
logits = h @ W_out + b    # predict next character from hidden state
```

The function `f` reads the previous hidden state and the current character's **embedding** (a learned vector that represents each character as a point in continuous space), and produces a new hidden state. This is fundamentally different from feedforward networks that process each input independently: the hidden state acts as the model's "memory" of everything it has read so far.

Making deep networks trainable

Simply stacking 32 layers of `h = ReLU(h @ W + b)` does not work: the hidden state either explodes to infinity or collapses to zero as it passes through many layers. Three techniques solve this:

- **Residual connections:** Instead of `h = f(h)`, compute `h = h + f(h)`. The identity "shortcut" ensures that gradients can flow backwards through the network even if `f` has very small gradients. This was the key innovation behind deep learning's leap from ~10 to ~100+ layer networks.
- **Normalisation (RMSNorm):** Before each layer, rescale the hidden state to have unit root-mean-square: `h / sqrt(mean(h²) + ε)`. This prevents the activations from growing exponentially across layers or time steps.
- **Input injection:** Add the character embedding at every layer (not just the first), so each layer can directly access the current input rather than relying on information that has been progressively transformed and potentially diluted by many preceding layers.

Training: learning by gradient descent

The weight matrices start with random values. **Training** means repeatedly: (1) run the model on a batch of text, (2) compute how wrong the predictions are (the loss), (3) compute the gradient of the loss with respect to every weight (using **backpropagation** — the chain rule applied systematically through the network), and (4) nudge each weight in the direction that reduces the loss. For recurrent networks, this is called **backpropagation through time (BPTT)** because gradients flow backwards through both layers and time steps.

Training typically happens on a GPU because GPUs have massive memory bandwidth and can process large batches in parallel. The NPU enters at **inference** time: once the weights are learned, we deploy the trained model to the NPU for fast, efficient text generation.

3. The Hardware

The AMD XDNA 2 NPU in the Ryzen AI 9 HX 370 (codename Strix Point) is a tiled spatial-dataflow processor with the following structure:

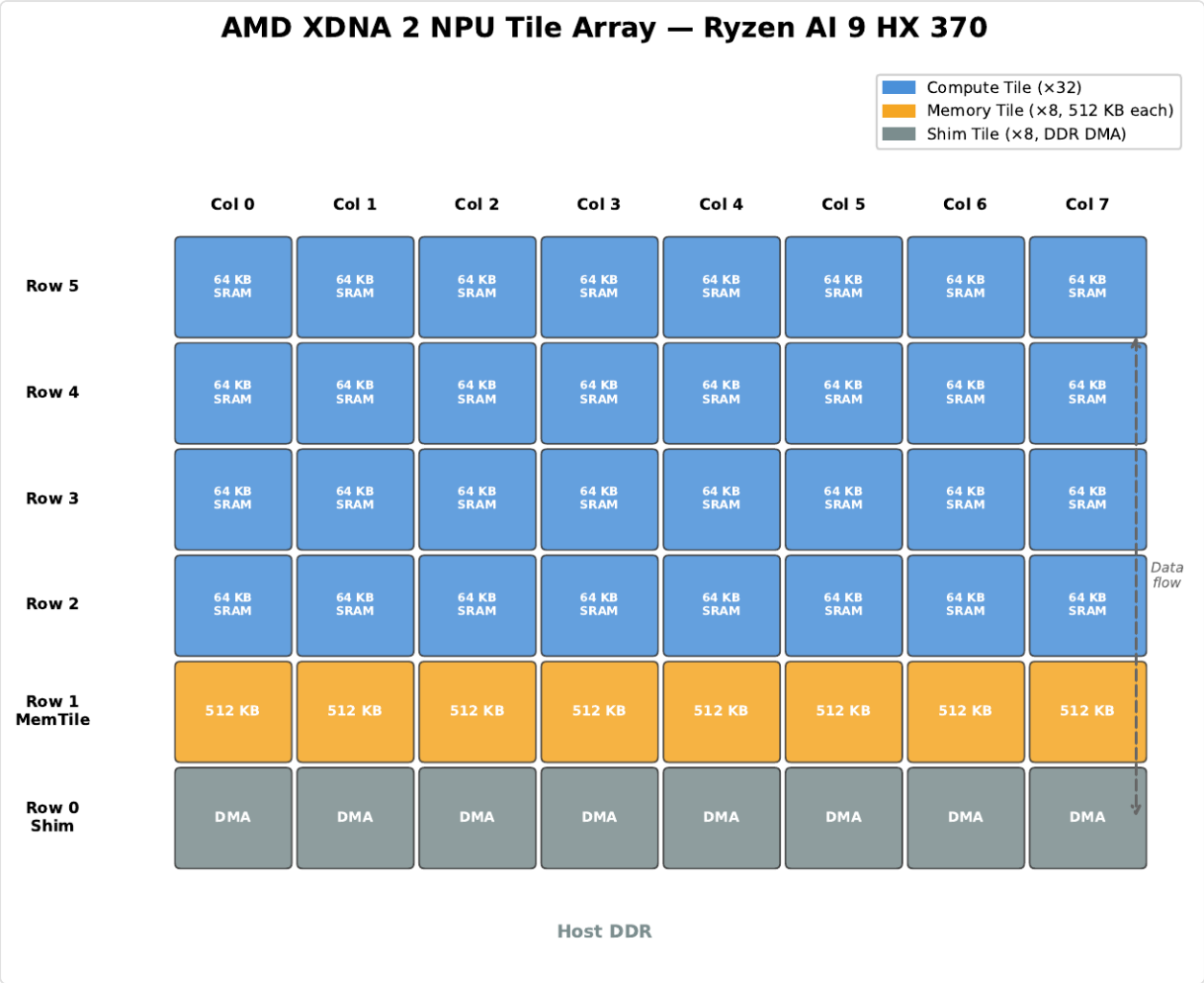

Figure 1: Physical tile array of the AMD XDNA 2 NPU. 32 compute tiles (rows 2–5) each contain ~64 KB SRAM and a bf16 MMUL unit. 8 memory tiles (row 1, 512 KB each) serve as on-chip L2 buffers and routing hubs. 8 shim tiles (row 0) provide DMA access to host DDR memory.

Property	Value
Compute tiles	32 (8 columns × 4 rows)

Memory tiles	8 (512 KB each, 4 MB total)
Per-tile SRAM	~64 KB data memory
Per-tile compute	bf16 MMUL unit, VLIW+SIMD core
Clock frequency	~1.5 GHz
Peak throughput	**25 TFLOPS** (bfloat16)
Interconnect	ObjectFIFOs (tile-to-tile double-buffered streams)
Power envelope	~6 W

3.1 Why spatial dataflow matters

On a CPU, a matrix multiply reads data from DRAM through multiple cache levels. Each layer of a neural network bounces activations through L1→L2→L3→DRAM and back. On the NPU, data stays in 64 KB tile SRAM between operations — zero cache misses, zero bus contention. This is the source of the NPU's advantage for deep, narrow computations.

> **Why on-chip matters:** A single large GEMM achieves only 2.49 TFLOPS on the NPU (10% of peak) because it is *memory-bandwidth limited* — data must stream from DDR. The NPU wins when data **stays on-chip**. Our architecture keeps weights in tile SRAM and passes activations tile-to-tile through FIFOs.

4. The Architecture: Block-Recurrent Character LM

We build a character-level language model whose architecture is dictated by the NPU's physical tile array. The model has 32 layers (one per compute tile), grouped into 8 blocks of 4 layers (one block per column). Each block maps to a single NPU pipeline call. Between blocks, the CPU applies the operations that the pipeline cannot: normalisation, embedding injection, and residual connections.

4.1 The block-recurrent computation

For each character in the input sequence, the model updates a hidden state `h` (a 128-dimensional bfloat16 vector) through 8 blocks:

```
for each character:
    for each block g = 0..7:
        CPU:  h_b = RMSNorm(h) + embed(char) + bias_g       ← prepare input
        NPU:  h_b = ReLU(  ReLU(  ReLU(  ReLU(
```

```
                  h_b @ W[4g]
              ) @ W[4g+1]
            ) @ W[4g+2]
          ) @ W[4g+3] )                                   ← 4-stage pipeline
        CPU:  h = h + h_b                                 ← residual connection
    CPU:  h = RMSNorm(h)                                  ← post-norm
    CPU:  logits = h @ W_out + b_out                      ← predict next char
```

Each of the 8 NPU calls runs a 4-stage matmul+ReLU pipeline:

1. The CPU prepares the block input: normalise the hidden state, add the character embedding and a per-block learned bias.
2. The prepared vector is sent to the NPU, where it passes through 4 tiles in sequence. Each tile multiplies by its own weight matrix W_i (128×128) and applies ReLU, using the fused `matmul_relu` kernel.
3. The NPU returns the result to the CPU, which adds it back to the hidden state (residual connection).

After all 8 blocks, a final RMSNorm and a linear readout produce a probability distribution over the ~65-character vocabulary.

4.2 NPU tile mapping

The model uses all 32 compute tiles in an 8-column × 4-row layout:

```
Column 0        Column 1        ...  Column 7
(samples 0-47)  (samples 48-95)      (samples 336-383)

  Row 2: W&sub0;       Row 2: W&sub0;    ...    Row 2: W&sub0;      Stage 1
  ↓              ↓                    ↓
  Row 3: W&sub1;       Row 3: W&sub1;    ...    Row 3: W&sub1;      Stage 2
  ↓              ↓                    ↓
  Row 4: W&sub2;       Row 4: W&sub2;    ...    Row 4: W&sub2;      Stage 3
  ↓              ↓                    ↓
  Row 5: W&sub3;       Row 5: W&sub3;    ...    Row 5: W&sub3;      Stage 4
```

- **8 columns** process 8 independent batch slices in parallel, 48 samples each = 384 total.
- **4 rows** per column form a pipeline: data flows from row 2 to row 5 through ObjectFIFOs, tile-to-tile, with no DDR traffic between stages.
- All 8 columns share the same 4 weight matrices for a given block. The CPU loads the appropriate block's weights before each NPU call.

4.3 Why this architecture

Every architectural choice follows from a hardware constraint:

- **H=128:** A 128×128 weight matrix occupies 32 KB in bf16. With two activation buffers (48×128 = 12 KB each, double-buffered) and ~1 KB of stack, the total is 57 KB — within the 64 KB per-tile SRAM limit.
- **4 layers per block:** The NPU has 4 compute rows. A 4-stage pipeline uses one row per stage, fully utilizing the vertical dimension.
- **8 blocks:** With 4 layers per block and 32 total layers, 8 blocks = 32 layers = 32 tiles. This is the maximum depth that maps cleanly to the hardware.
- **B=48:** The fused `matmul_relu` kernel pipelines efficiently only when the outer loop has ≥3 iterations: M/(2r) ≥ 3, so B ≥ 48 for r = 8. (See `logbook.md` for the kernel fusion analysis.)
- **384 parallel sequences:** 8 columns × 48 samples per column. Processing many sequences in parallel amortises the ~120 µs XRT driver overhead per NPU call.

4.4 The hardware–quality trade-off

Our model architecture is shaped by a fundamental constraint: the NPU pipeline executes a pure chain of ReLU(x × W) operations with no CPU intervention between stages. But the best-quality version of the model applies normalisation and input injection at *every* layer — operations that require CPU processing between each matmul.

This creates a trade-off:

Model variant	Val Loss	Perplexity	NPU-compatible?
Per-layer (norm + embed every layer)	1.94	6.9	No — CPU needed between every matmul
Blocked (4-layer pure groups)	**2.42**	**11.2**	**Yes — exact NPU mapping**

The blocked model sacrifices quality (perplexity 11.2 vs 6.9) for exact hardware mapping. Within each block, the 4-stage matmul+ReLU chain executes entirely in tile SRAM with no DDR traffic. Between blocks, the CPU handles the operations that need it: normalisation (RMSNorm), character embedding injection, bias addition, and residual connections.

Design principle: We train the model with exactly the same block structure that runs on the NPU. There is no “compilation” step that approximates a richer model for hardware. What you train is what you deploy.

4.5 Making it trainable

Simply stacking 32 layers of ReLU(h × W + b) does not work: the hidden state either explodes to infinity or collapses to zero. We use three techniques (explained in Section 2.7):

- **Pre-block RMSNorm:** Before each block, normalise the hidden state to unit RMS. This prevents the activations from growing exponentially across the 8 blocks and across the 64-character sequence during backpropagation through time.
- **Residual connections:** `h = h + block(h)` after each block. Gradients flow backwards through the identity shortcut, keeping training stable even at 32 layers.
- **Input injection:** The character embedding is added at the start of every block, not just the first. This ensures each block can directly access the current input, rather than relying on information that has been progressively transformed by preceding blocks.
- **Spectral norm clamping:** During training, each weight matrix is periodically rescaled so its largest singular value ≤ 1. This prevents any single layer from amplifying the hidden state, keeping the recurrent dynamics stable.

5. Results

5.1 Language model quality

The block-recurrent model was trained on the tiny Shakespeare dataset (1.1 MB, ~1.1M characters) for 10 epochs on a GPU, then deployed to the NPU:

Model	Parameters	Val Loss	Perplexity	Device
Block-recurrent (8×4 pipeline)	**542K**	**2.42**	**11.2**	**NPU (32 tiles)**
Per-layer recurrent (norm every layer)	542K	1.94	6.9	CPU/GPU only
Transformer baseline (GPT-style)	818K	1.89	6.6	GPU

The per-layer model and the transformer baseline serve as quality references. The blocked model produces recognisable Shakespearean English, though with occasional incoherence — expected at perplexity 11.2 with a 542K-parameter model trained on 1 MB of text.

5.2 NPU inference performance

Metric	Value
NPU tiles used	32 (8 columns × 4 rows)
NPU calls per character	8 (one per block)
Latency per NPU call	~0.14 ms
Total NPU time per character	~1.1 ms
Total time per character (NPU + CPU)	~4.2 ms
Throughput (1 sequence)	233 chars/s
Throughput (384 parallel sequences)	**89,600 chars/s**

The 384-sequence throughput is the key figure: by processing 8 columns × 48 samples per column in parallel, we amortise the per-call overhead across hundreds of sequences. The CPU overhead between NPU calls (RMSNorm, embedding lookup, residual addition) accounts for the gap between the 1.1 ms of NPU time and the 4.2 ms total latency per character.

5.3 What limits throughput

Each NPU call performs only 4 matmuls of size 48×128×128 — a tiny amount of arithmetic completed in microseconds. The ~0.14 ms measured per call is dominated by XRT driver overhead (instruction dispatch, DMA setup), not compute. With 8 calls per character, that overhead adds up.

This is fundamentally different from the throughput benchmark (see `logbook.md`), where a single NPU call ran 1000 loop iterations on-chip, making compute dominate overhead and achieving 95.7% of the 25 TFLOPS peak. The character LM trades peak utilisation for a richer, more useful architecture.

> **Insight:** NPU utilisation is not the goal — building a useful model that runs on the NPU is. The architecture is designed for correctness of the hardware mapping; throughput optimisation (larger hidden dimensions, fewer blocks, quantisation) is future work.

6. The Toolchain

Component	Role

IRON	Python API for tile layout, ObjectFIFOs, and dataflow
MLIR-AIE	MLIR dialect → hardware bitstream compilation
Peano/LLVM-AIE	C++ compiler for per-tile kernels
XRT	Runtime for loading and executing on the NPU

6.1 Compilation pipeline

```
design.py  →  MLIR  →  aiecc  →  .xclbin (bitstream)
                          →  .bin   (instruction sequence)

matmul_relu.cc →  mlp_mm_relu.o →  mlp_kernels.a
mlp_kernels.cc →  mlp_copy.o    ↗
```

7. Code Structure

The project is intentionally minimal — four Python files and two C++ files:

File	Lines	Purpose
`char_lm/model.py`	~210	Block-recurrent character LM (RecurrentCharLM)
`char_lm/train.py`	~250	GPU training loop (ROCm / CUDA), spectral norm clamping
`char_lm/generate.py`	~215	Text generation on CPU or NPU
`char_lm/data.py`	~80	Shakespeare dataset and vocabulary
`char_lm/transformer_baseline.py`	~240	GPT-style reference model for quality comparison
`spatial_mlp/__init__.py`	~55	Tiling utilities (`to_tiled` , `from_tiled`)
`spatial_mlp/pipeline_design.py`	~290	IRON design: 32-tile pipeline (8 cols × 4 rows)
`spatial_mlp/pipeline_op.py`	~130	IRON operator: compilation + runtime buffers
`aie_kernels/matmul_relu.cc`	~125	Fused C = ReLU(A × B) kernel for AIE2P
`aie_kernels/mlp_kernels.cc`	~55	Support kernel: `copy_bf16` for result staging

The character LM module (`char_lm/`) handles training and generation. The spatial MLP module (`spatial_mlp/`) provides the NPU pipeline infrastructure used by the generator for on-device inference.

8. Future Work

- **Close the quality gap:** The blocked model (val loss 2.42) lags the per-layer model (1.94) because normalization and input injection happen only between blocks, not between every layer. Custom NPU kernels that fuse norm+matmul+ReLU could enable per-layer operations within the pipeline, recovering quality without sacrificing NPU mapping.
- **INT8 mode:** The NPU's peak is 50 TOPS for int8 — double the bf16 rate. With quantization-aware training, this could push effective throughput to ~48 TOPS and allow H=256 (fitting in the same SRAM budget).
- **Training on NPU:** Research NPU backpropagation (see arXiv:2504.03083). The block structure could allow per-block gradient computation on-chip, with CPU handling the cross-block gradient flow.
- **Larger tasks:** Apply the block-recurrent architecture to longer text datasets, music generation, or time-series forecasting where deep recurrence is natural and the NPU throughput advantage compounds over long sequences.

References

1. AMD IRON repository: github.com/amd/IRON
2. MLIR-AIE programming guide: github.com/Xilinx/mlir-aie
3. IRON tutorial (IPDPS 2025): AMD Technical Paper
4. NPU training: arXiv:2504.03083
5. Linux kernel NPU docs: kernel.org